

# Task 1: Simulation and Sensitivity Analysis of Chaotic Systems

Hassan Darwish

This report covers the three parts of the task: discrete chaotic maps, continuous chaotic and hyperchaotic flows, and a sensitivity analysis. Every figure title carries the exact settings used, and the settings tables repeat them so the results can be reproduced. The code is in the accompanying notebook.

## Part A — Discrete Chaotic Systems

### A.2 Logistic Map

The map I iterated:

$$x_{n+1} = r \cdot x_n \cdot (1 - x_n)$$

|                          |                                              |
|--------------------------|----------------------------------------------|
| <b>State variable</b>    | x (scalar)                                   |
| <b>Control parameter</b> | r = 3.9 (chaotic range)                      |
| <b>Initial condition</b> | $x_0 = 0.5$                                  |
| <b>Iterations</b>        | 1500 total, first 500 discarded as transient |

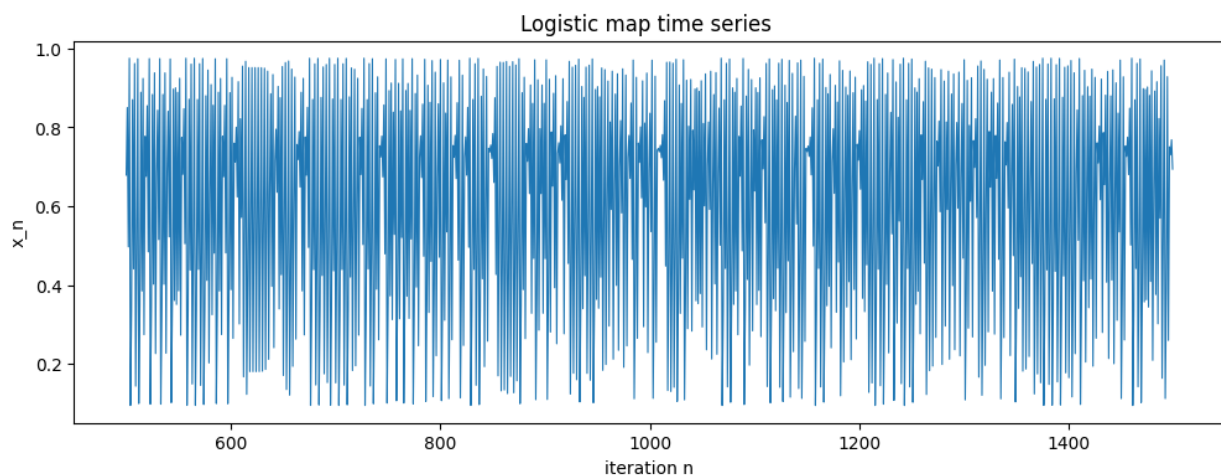


Figure 1 — Logistic map time series at  $r = 3.9$ . Iterations 500–1499 shown.

At  $r = 3.9$  the values never settle and never repeat. They keep bouncing around inside roughly  $[0.1, 1.0]$  with no pattern I can pick out by eye, which is what chaos looks like for this map. The rule generating this is one line of arithmetic with no randomness in it at all — the irregularity comes entirely from the nonlinearity.

### A.3 Hénon Map

The two equations:

$$\begin{aligned}x_{n+1} &= 1 - a \cdot x_n^2 + y_n \\ y_{n+1} &= b \cdot x_n\end{aligned}$$

Both equations read the state at  $n$ , so I compute the new  $x$  and the new  $y$  from the old pair and only then commit them. Updating  $x$  first and feeding it into  $y$  would be a different map.

|                           |                                              |
|---------------------------|----------------------------------------------|
| <b>State variables</b>    | $(x, y)$                                     |
| <b>Control parameters</b> | $a = 1.4, b = 0.3$ (classic chaotic values)  |
| <b>Initial condition</b>  | $(x_0, y_0) = (0, 0)$                        |
| <b>Iterations</b>         | 5500 total, first 500 discarded as transient |

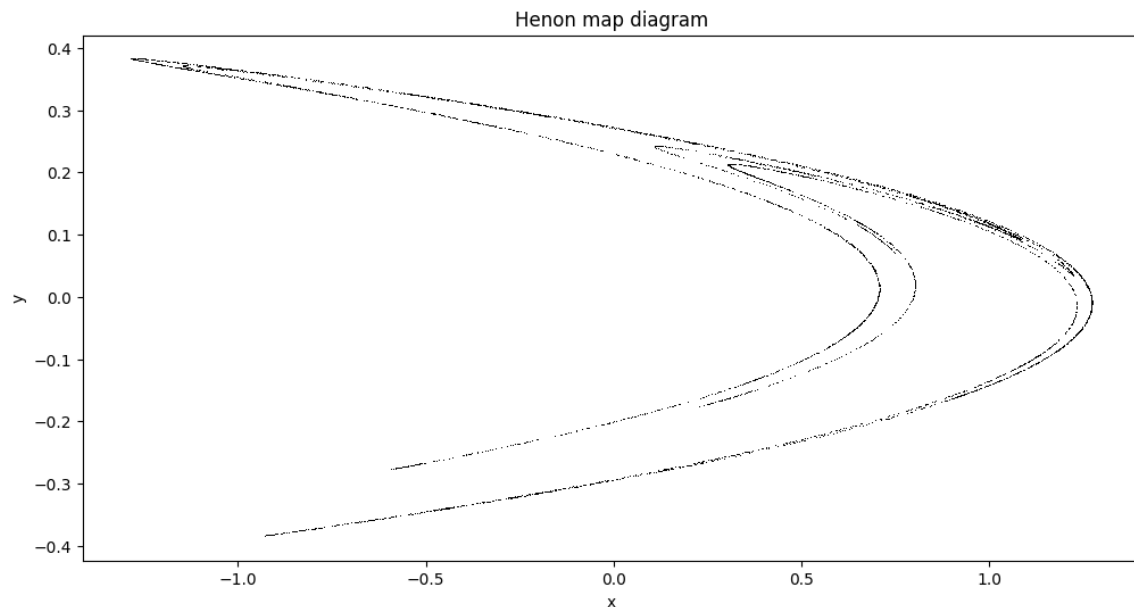


Figure 2 — Hénon attractor,  $a = 1.4, b = 0.3$ . 5000 retained iterates.

The points don't fill the plane and they don't collapse to a few spots either — they land on this bent, boomerang-shaped set. Zooming into any of the bands shows it's actually several finer strands lying next to each other, which is the fractal structure the stretching-and-folding of the map produces.

## A.4 Bifurcation Diagrams

For both diagrams I swept the control parameter, ran a fresh simulation from the same initial condition at every parameter value, threw away the transient, and plotted only the iterates that survived. Each vertical slice is therefore an independent run showing where that parameter value ends up in the long term.

|                       |                                                                                          |
|-----------------------|------------------------------------------------------------------------------------------|
| <b>Logistic sweep</b> | $r \in [2.5, 4.0]$ , step 0.003; 1500 iterations, 500 discarded; $x_0 = 0.5$             |
| <b>Hénon sweep</b>    | $a \in [1.0, 1.4]$ , step 0.003, $b = 0.3$ ; 5500 iterations, 500 discarded; IC = (0, 0) |

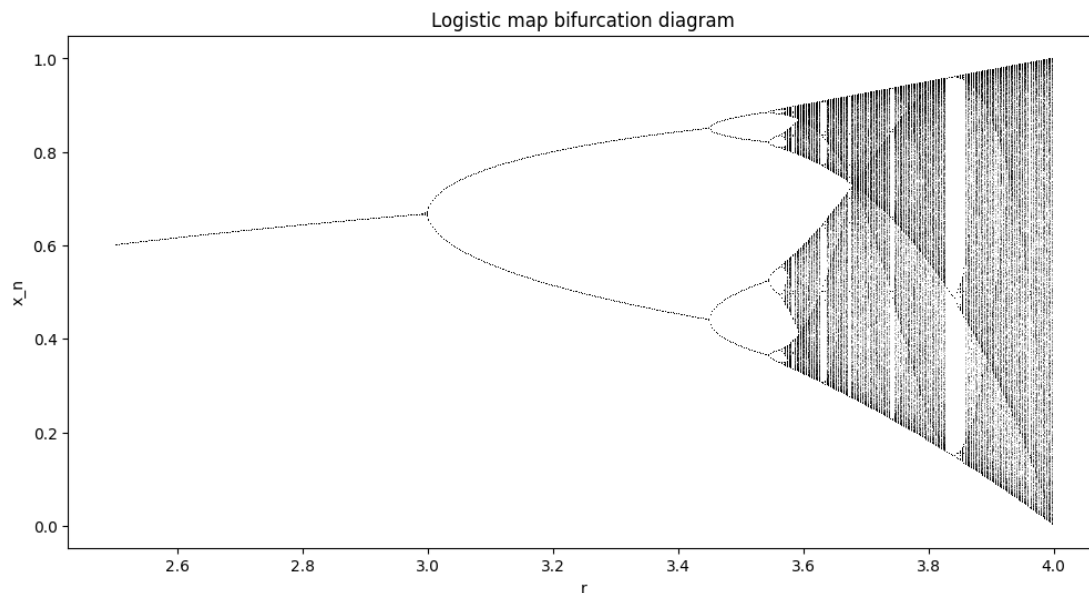


Figure 3 — Logistic map bifurcation diagram,  $r \in [2.5, 4.0]$ ,  $x_0 = 0.5$ , 1500 iterations, 500 discarded.

Up to  $r \approx 3$  there is one line, meaning the system settles on a single fixed value. At  $r \approx 3$  it splits into two, then into four at  $r \approx 3.45$ , then eight, and the splits keep coming faster and faster until around  $r \approx 3.57$  where the plot goes dark and filled — that is chaos. The clean vertical gaps inside the dark region are periodic windows, where the system briefly becomes regular again before going back to chaos. This whole period-doubling cascade is the route to chaos for this map.

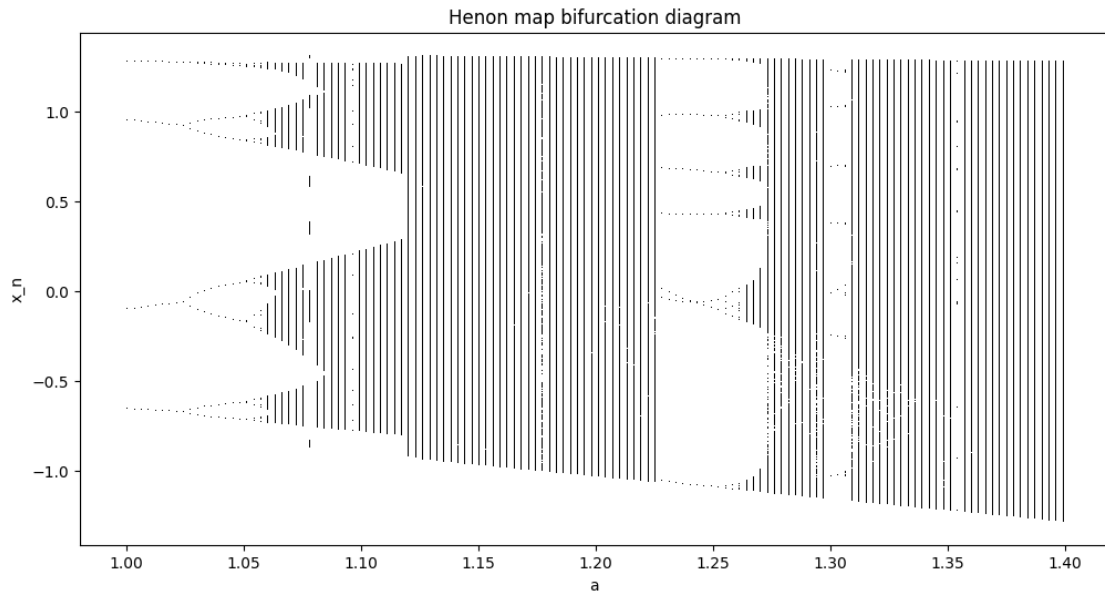


Figure 4 — Hénon map bifurcation diagram,  $a \in [1.0, 1.4]$ ,  $b = 0.3$ , 5500 iterations, 500 discarded.

Across almost the whole range I swept there are two branches, so the system is sitting on a period-2 orbit and just drifting as  $a$  changes. The interesting part is squeezed into the last stretch: around  $a \approx 1.38$  each branch splits again, and by  $a = 1.4$  it has broken up into the chaotic scatter. That  $a = 1.4$  endpoint is exactly the value I used in Figure 2, so the diagram confirms the attractor I plotted is a chaotic one and not a periodic orbit.

## Part B — Continuous Chaotic Systems

These are differential equations, not maps — they give the derivative, not the next state. So I can't just loop the equation; I need a numerical integrator to turn the derivatives into a trajectory. For this part I used scipy's solve\_ivp with RK45 and tight tolerances (rtol = atol = 1e-9). The tolerances matter more than the output spacing here, because RK45 picks its own internal step and dt only controls where the solution is sampled.

### B.2 Lorenz System

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z\end{aligned}$$

|                          |                                           |
|--------------------------|-------------------------------------------|
| <b>State variables</b>   | (x, y, z)                                 |
| <b>Parameters</b>        | $\sigma = 10, \rho = 28, \beta = 8/3$     |
| <b>Initial condition</b> | (1, 1, 1)                                 |
| <b>Output step, dt</b>   | 0.01                                      |
| <b>Simulation length</b> | T = 50                                    |
| <b>Solver</b>            | scipy solve_ivp, RK45, rtol = atol = 1e-9 |

Lorenz attractor ( $\sigma=10.0$ ,  $\rho=28.0$ ,  $\beta=8/3$ , IC=(1,1,1), dt=0.01, T=50, RK45)

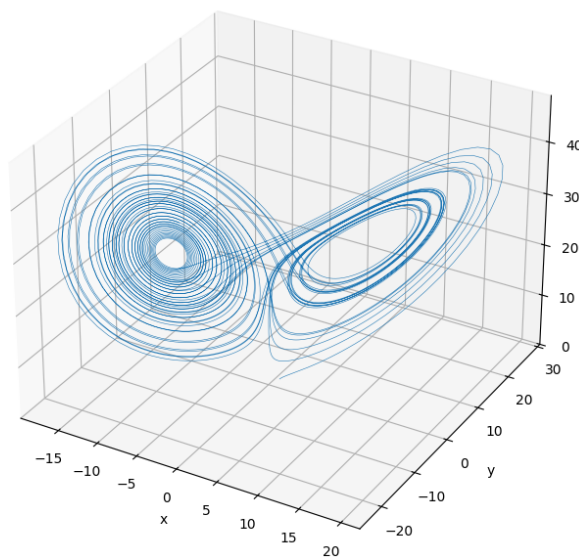


Figure 5 — Lorenz attractor in phase space.

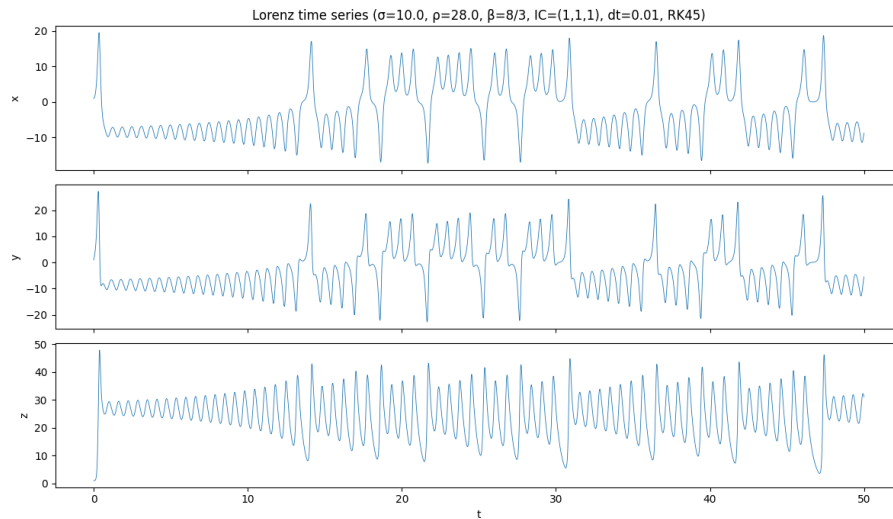


Figure 6 — Lorenz time series,  $x(t)$ ,  $y(t)$ ,  $z(t)$ .

The trajectory winds around one lobe for a while, jumps to the other, and keeps swapping at irregular intervals — that's the butterfly. In the time series you can see  $x$  and  $y$  flip sign at every one of those jumps while  $z$  stays positive and just pulses. The trajectory never closes on itself but it never escapes the attractor either.

## B.3 Rössler System

$$\begin{aligned}\dot{x} &= -y - z \\ \dot{y} &= x + a \cdot y \\ \dot{z} &= b + z(x - c)\end{aligned}$$

|                          |                                           |
|--------------------------|-------------------------------------------|
| <b>State variables</b>   | (x, y, z)                                 |
| <b>Parameters</b>        | a = 0.2, b = 0.2, c = 5.7                 |
| <b>Initial condition</b> | (1, 1, 1)                                 |
| <b>Output step, dt</b>   | 0.01                                      |
| <b>Simulation length</b> | T = 300                                   |
| <b>Solver</b>            | scipy solve_ivp, RK45, rtol = atol = 1e-9 |

Rössler attractor (a=0.2, b=0.2, c=5.7, IC=(1,1,1), dt=0.01, T=300, RK45)

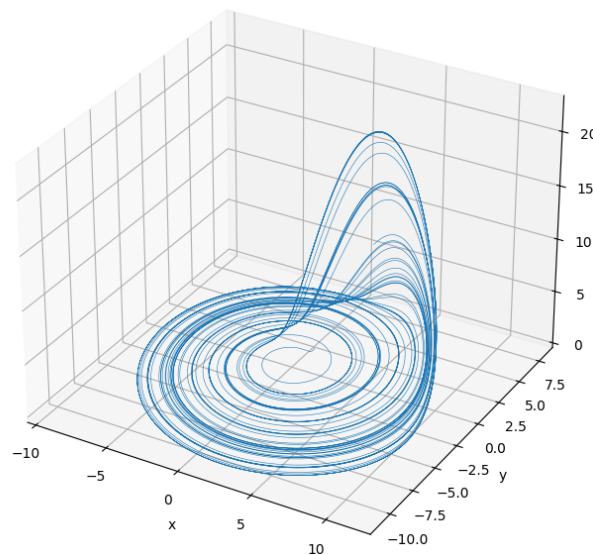


Figure 7 — Rössler attractor in phase space.

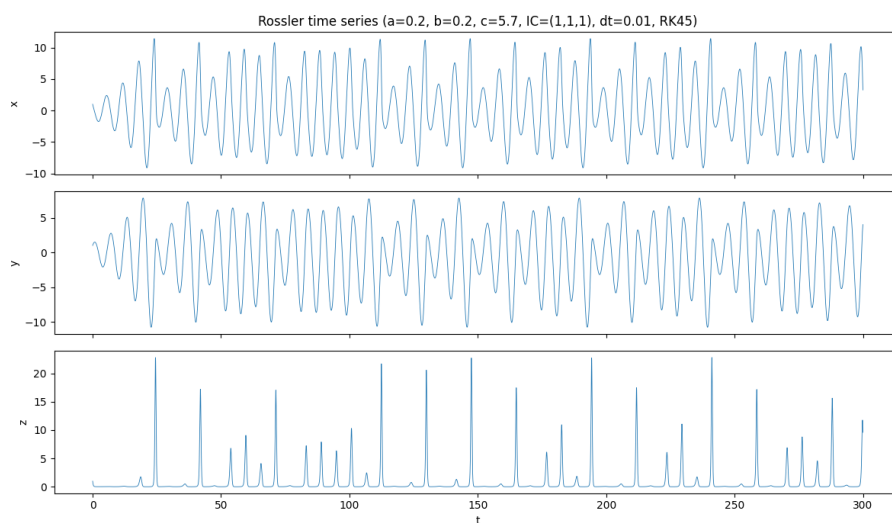


Figure 8 — Rössler time series,  $x(t)$ ,  $y(t)$ ,  $z(t)$ .

Rössler only has one scroll instead of Lorenz's two. The trajectory spirals outwards in a flat sheet, and when it gets far enough out the  $z$  equation kicks in and throws it up and back into the middle. That shows up in the time series as  $z$  sitting near zero most of the time and then

spiking sharply — one spike per fold. Only one of the three equations is nonlinear here, which makes this the simplest of the three flows.



## B.4 Chen System

$$\begin{aligned}\dot{x} &= a(y - x) \\ \dot{y} &= (c - a)x - xz + cy \\ \dot{z} &= xy - bz\end{aligned}$$

|                          |                                           |
|--------------------------|-------------------------------------------|
| <b>State variables</b>   | (x, y, z)                                 |
| <b>Parameters</b>        | a = 35, b = 3, c = 28                     |
| <b>Initial condition</b> | (-0.1, 0.5, -0.6)                         |
| <b>Output step, dt</b>   | 0.003                                     |
| <b>Simulation length</b> | T = 50                                    |
| <b>Solver</b>            | scipy solve_ivp, RK45, rtol = atol = 1e-9 |

Chen attractor (a=35, b=3, c=28, IC=(-0.1,0.5,-0.6), dt=0.003, T=50, RK45)

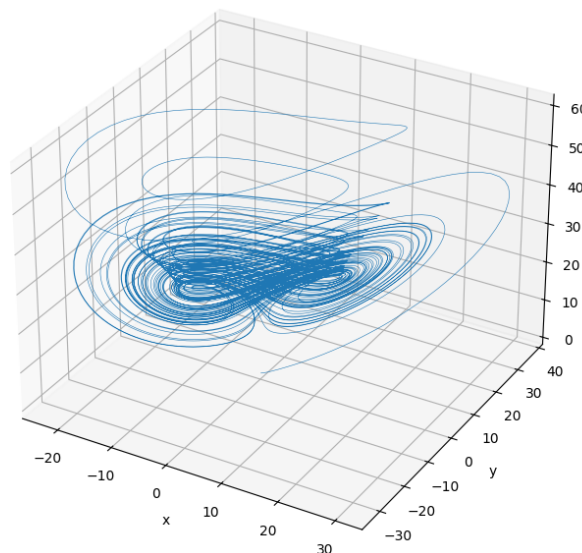


Figure 9 — Chen attractor in phase space.

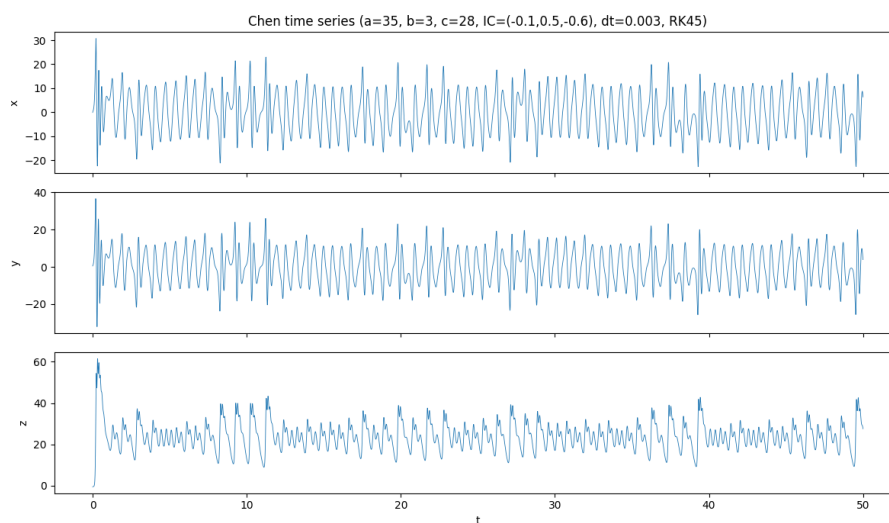


Figure 10 — Chen time series,  $x(t)$ ,  $y(t)$ ,  $z(t)$ .

Chen is a double-scroll like Lorenz but the two lobes sit at a different angle to each other and the whole thing is more tightly wound. It's also noticeably stiffer to integrate:  $a = 35$  against Lorenz's  $\sigma = 10$  means the contraction term is about 3.5 times faster, so the solver has to take

much smaller internal steps to keep up. Same tolerances as Lorenz took considerably longer to run.

## B.5 Hyperchaotic Rössler System

$$\begin{aligned}\dot{x} &= -y - z \\ \dot{y} &= x + a \cdot y + w \\ \dot{z} &= b + xz \\ \dot{w} &= -c \cdot z + d \cdot w\end{aligned}$$

|                          |                                           |
|--------------------------|-------------------------------------------|
| <b>State variables</b>   | (x, y, z, w)                              |
| <b>Parameters</b>        | a = 0.25, b = 3, c = 0.5, d = 0.05        |
| <b>Initial condition</b> | (-10, -6, 0, 10)                          |
| <b>Output step, dt</b>   | 0.01                                      |
| <b>Simulation length</b> | T = 250                                   |
| <b>Solver</b>            | scipy solve_ivp, RK45, rtol = atol = 1e-9 |

This one is 4-D so there's no single phase-space plot that shows everything. I plotted three 2-D projections plus an (x, y, z) projection to compare against the plain Rössler in Figure 7.

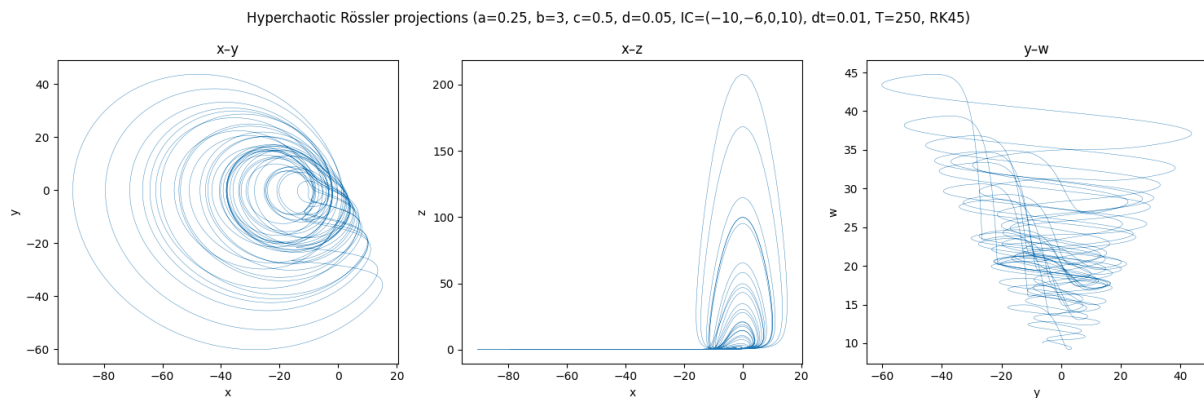


Figure 11 — Hyperchaotic Rössler, 2-D projections (x-y, x-z, y-w).

Hyperchaotic Rössler, (x,y,z) projection (a=0.25, b=3, c=0.5, d=0.05, IC=(-10,-6,0,10), dt=0.01, T=250, RK45)

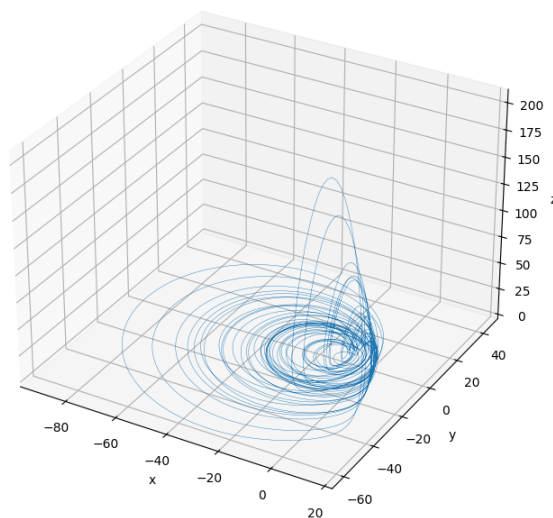


Figure 12 — Hyperchaotic Rössler, (x, y, z) projection.

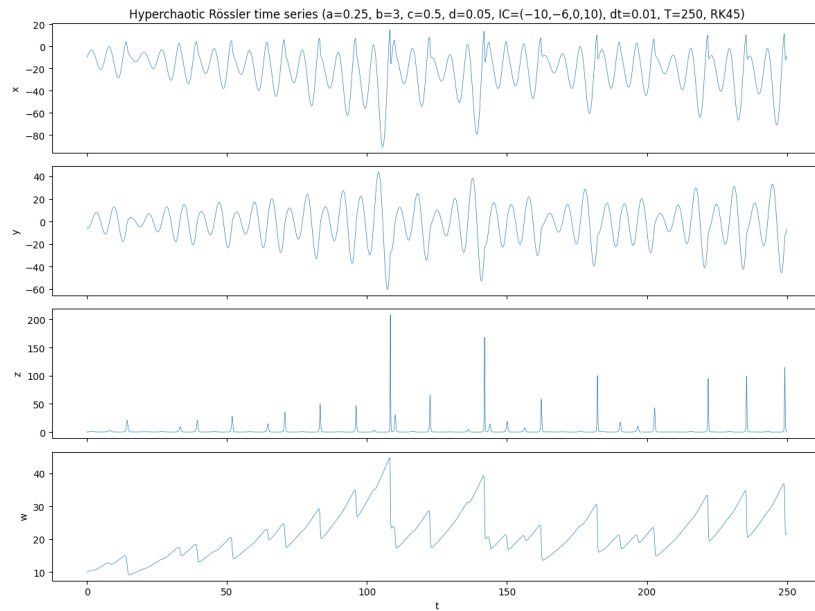


Figure 13 — Hyperchaotic Rössler time series, all four states.

The extra  $w$  variable is what makes this hyperchaotic rather than chaotic. In the projections the trajectory doesn't stay on a thin sheet the way plain Rössler does — it spreads out into a thicker, more filled-in region, because it is now stretching in two directions at once instead of one. The time series makes the point clearly:  $x$  and  $y$  oscillate fast,  $z$  sits near zero and spikes, but  $w$  drifts on a much slower timescale and keeps pushing the whole system around. That mismatch in timescales is the second positive Lyapunov exponent showing up visually.

## Part C — Sensitivity Analysis

### C.1 Parameter Sweep of the Lorenz System

This is the continuous version of the bifurcation diagrams in Part A. For each  $\rho$  I integrated the full system, threw away the first 20 time units, and then recorded the local maxima of  $z(t)$  — the points where  $z[i-1] < z[i] > z[i+1]$ . Plotting every  $z$  value would just give a solid vertical smear because  $z$  is continuous, so taking only the peaks samples the trajectory once per orbit, which is what makes it comparable to one iterate of a discrete map.

|                          |                                                                      |
|--------------------------|----------------------------------------------------------------------|
| <b>Swept parameter</b>   | $\rho \in [0, 30]$ , step 0.2 ( $\sigma = 10$ , $\beta = 8/3$ fixed) |
| <b>Initial condition</b> | (1, 1, 1)                                                            |
| <b>Output step, dt</b>   | 0.01                                                                 |
| <b>Simulation length</b> | $T = 100$ per $\rho$ , first 20 time units discarded                 |
| <b>Recorded quantity</b> | local maxima of $z(t)$                                               |
| <b>Solver</b>            | scipy solve_ivp, RK45, $\text{rtol} = \text{atol} = 1\text{e-}9$     |

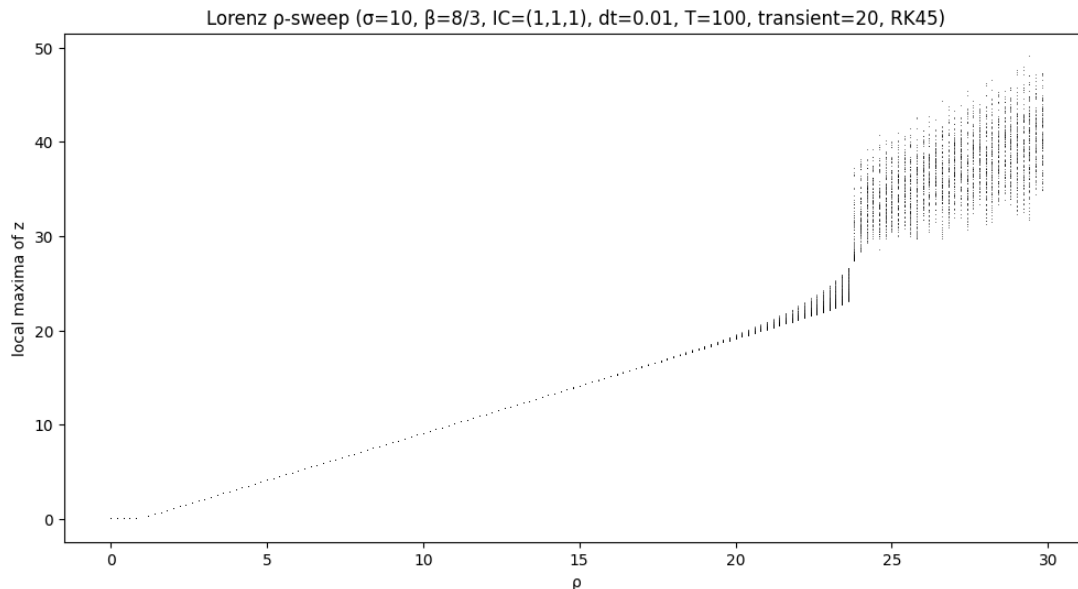


Figure 14 — Lorenz  $\rho$ -sweep. Local maxima of  $z$  against  $\rho$ .

Below  $\rho \approx 1$  everything sits at zero — the origin is stable and  $z$  just decays, so there is nothing to record. From  $\rho \approx 1$  up to about 23 there's a single rising line: the system settles onto a fixed point, so every orbit gives the same maximum. Between roughly 23 and 24.7 short vertical stripes appear, which is transient chaos — the fixed points are still stable but the trajectory wanders for a long time before finding them. Past  $\rho \approx 24.7$  the plot breaks into a dense scattered band and stays there, and that's the onset of proper chaos, which matches the textbook value of about 24.74. The thing worth noticing is that Lorenz does not period-double its way in like the logistic map did — the fixed point loses stability abruptly and chaos appears all at once, so this diagram looks very different from Figure 3 even though both are showing the same kind of transition.

## C.2 Step-Size Sensitivity

For this part I could not use `solve_ivp`. RK45 is adaptive — it picks its own internal step and `dt` only controls where the output is sampled, so all three runs would have come out essentially identical and there would be nothing to compare. I wrote a fixed-step RK4 integrator instead, which calls the same Lorenz derivative function four times per step and averages them. That way `dt` genuinely drives the integration and the comparison means something.

|                          |                                                                   |
|--------------------------|-------------------------------------------------------------------|
| <b>Parameters</b>        | $\sigma = 10, \rho = 28, \beta = 8/3$                             |
| <b>Initial condition</b> | (1, 1, 1) — identical for all three runs                          |
| <b>Step sizes</b>        | $dt = 0.001$ (fine), $dt = 0.01$ (baseline), $dt = 0.05$ (coarse) |
| <b>Simulation length</b> | $T = 50$                                                          |
| <b>Solver</b>            | fixed-step RK4 (own implementation)                               |

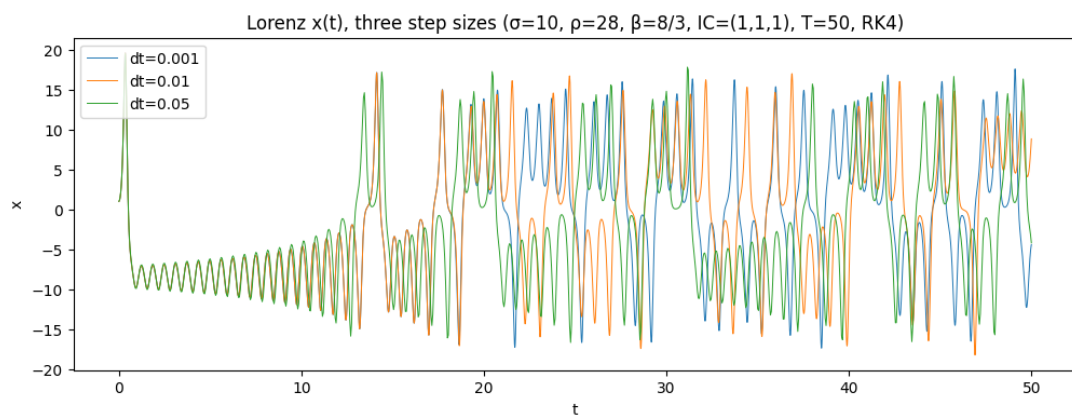


Figure 15 — Lorenz  $x(t)$  at three step sizes, overlaid.

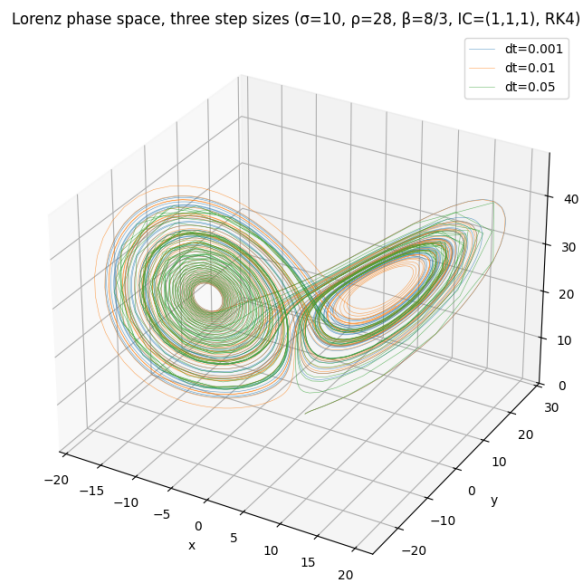


Figure 16 — Lorenz phase-space trajectories at three step sizes, overlaid.

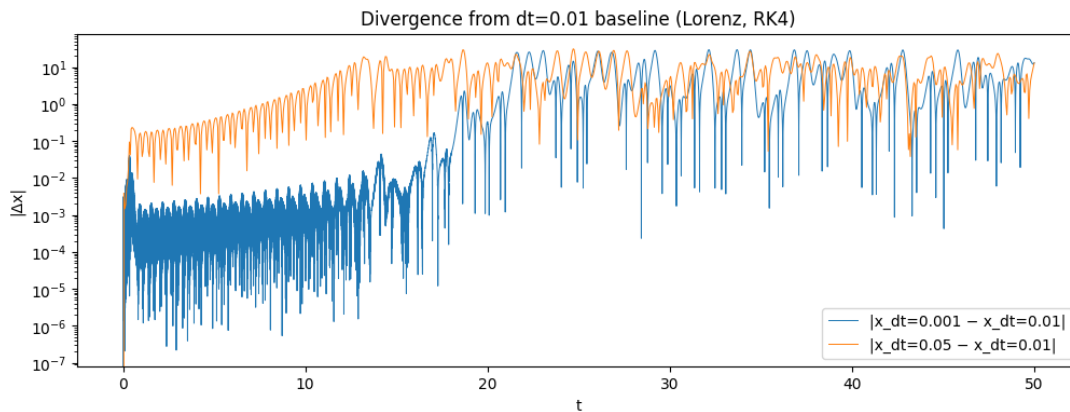


Figure 17 — Divergence from the  $dt = 0.01$  baseline, log scale.

All three runs start from exactly the same initial condition and follow each other closely for about the first 20 time units, then separate completely and go their own ways. Nothing was changed between them except the step size, so the difference starts as pure numerical error — but the system amplifies it. The divergence plot shows this best: on a log axis the error climbs in a roughly straight line from around  $1e-8$  up to order 10, which means it is growing exponentially, and the slope of that line is essentially the largest Lyapunov exponent. Once the error reaches the size of the attractor itself it flattens off, because the two trajectories are now just wandering around the same object independently and can't get any further apart.  $dt = 0.05$  breaks away earliest, as expected from the coarsest step.

The important thing is what the phase-space plot shows: all three trajectories trace the same butterfly. The specific path is not reproducible past  $t \approx 20$ , but the shape of the attractor is. So the pointwise prediction is lost while the statistical structure survives. On which  $dt$  is adequate — 0.05 is too coarse, it visibly distorts the trajectory early on. 0.01 resolves the attractor fine and is what I would use, and 0.001 is more accurate per step but costs ten times the compute for a trajectory that is going to diverge anyway.

## Conclusion

Discrete versus continuous. The maps in Part A were a single line of arithmetic in a loop — the map is the update rule, so there was nothing to solve. The flows in Part B gave me derivatives instead, which meant every one of them needed an integrator before I could plot anything, and the choice of integrator and tolerance became part of the result rather than an implementation detail. The other difference is dimension. The logistic map is 1-D and still chaotic, but a continuous system needs at least three dimensions before chaos is even possible, because a trajectory in a plane can't cross itself. That's why every flow in Part B is 3-D or bigger while the logistic map gets away with one variable.

What was the same in both was the behaviour. Bounded, never repeating, and completely determined by the initial state with no randomness anywhere in the code. The bifurcation diagrams and the  $p$ -sweep both showed the same idea — turn one knob, and the long-term behaviour goes from a fixed point to something periodic to chaos — even though the logistic map got there by period-doubling and Lorenz got there abruptly at  $p \approx 24.7$ .

Chaotic versus hyperchaotic. Comparing Figure 7 with Figures 11–13 is the clearest version of this. Plain Rössler stretches in one direction and stays on a thin sheet — one positive Lyapunov exponent. The hyperchaotic version adds a fourth variable, and the trajectory spreads into a thicker region because it's now stretching in two directions at once. The time series showed  $w$  drifting on a much slower timescale than  $x$  and  $y$  while still driving them, which is the visible signature of the second positive exponent. Three variables is enough for

chaos; hyperchaos needs four, and I could see why in the plots — the extra expanding direction has to live somewhere.

Part C tied it together. Sensitive dependence isn't only about the initial condition — in C.2 the initial conditions were identical and just changing the step size was enough to produce a completely different trajectory within 20 time units. Any perturbation will do, including one that's purely numerical. But the attractor itself was reproducible across all three runs, which is the practical takeaway: you can't predict where a chaotic system will be, but you can describe the set it will be somewhere in.